

CASE Expressions in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [Theory: Conditional Logic in SQL](#)
 3. [Simple CASE Expression](#)
 4. [Searched CASE Expression](#)
 5. [CASE in SELECT](#)
 6. [CASE in WHERE and HAVING](#)
 7. [CASE in ORDER BY](#)
 8. [CASE in GROUP BY](#)
 9. [CASE in Aggregate Functions](#)
 10. [CASE for Pivot Tables](#)
 11. [Nested CASE](#)
 12. [ASCII Visual Diagrams](#)
 13. [COALESCE, NULLIF, GREATEST, LEAST](#)
 14. [Common Mistakes](#)
 15. [Best Practices](#)
 16. [Performance Considerations](#)
 17. [Interview Questions & Answers](#)
 18. [Hands-on Exercises](#)
 19. [Advanced Notes](#)
 20. [Cross-references](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Write simple and searched CASE expressions
 - Use CASE for conditional aggregation and pivot tables
 - Apply CASE in ORDER BY for custom sort logic
 - Use COALESCE, NULLIF, GREATEST, and LEAST
 - Avoid CASE anti-patterns that hurt readability and performance
-

Theory: Conditional Logic in SQL

CASE is the SQL equivalent of if-then-else logic. It evaluates conditions and returns the first matching result. CASE is an expression (returns a value), not a statement (unlike PL/pgSQL IF).

Forms

```
1. Simple CASE:
CASE input_value
  WHEN match1 THEN result1
  WHEN match2 THEN result2
  ...
  ELSE default_result
END
```

2. Searched CASE:

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ...
  ELSE default_result
END
```

Key Properties

- Returns the result of the FIRST matching WHEN clause
 - If no WHEN matches and no ELSE, returns NULL
 - All THEN/ELSE values must be type-compatible
 - Can be used anywhere an expression is valid
-

Simple CASE Expression

Tests a single expression against a series of values (equality-only).

Examples

```
-- Example 1: Map department codes to names
```

```
SELECT
  employee_name,
  department_id,
  CASE department_id
    WHEN 1 THEN 'Engineering'
    WHEN 2 THEN 'Marketing'
    WHEN 3 THEN 'Sales'
    WHEN 4 THEN 'HR'
    ELSE 'Unknown'
  END AS department_name
FROM employees;
```

```
-- Example 2: Day of week label
```

```
SELECT
  order_id,
  order_date,
  CASE EXTRACT(DOW FROM order_date)
    WHEN 0 THEN 'Sunday'
    WHEN 1 THEN 'Monday'
    WHEN 2 THEN 'Tuesday'
    WHEN 3 THEN 'Wednesday'
    WHEN 4 THEN 'Thursday'
    WHEN 5 THEN 'Friday'
    WHEN 6 THEN 'Saturday'
  END AS day_of_week
FROM orders;
```

```
-- Example 3: Status code to description
```

```

SELECT
  order_id,
  CASE status_code
    WHEN 'P' THEN 'Pending'
    WHEN 'C' THEN 'Completed'
    WHEN 'X' THEN 'Cancelled'
    WHEN 'R' THEN 'Refunded'
    ELSE 'Unknown Status: ' || status_code
  END AS status_description
FROM orders;

```

Searched CASE Expression

Tests a series of boolean conditions — the most flexible form.

Examples

```

-- Example 4: Salary bands
SELECT
  employee_name,
  salary,
  CASE
    WHEN salary < 50000 THEN 'Junior'
    WHEN salary < 80000 THEN 'Mid-Level'
    WHEN salary < 120000 THEN 'Senior'
    WHEN salary < 200000 THEN 'Principal'
    ELSE 'Executive'
  END AS salary_band
FROM employees;

-- Example 5: Sales performance rating
SELECT
  rep_name,
  SUM(amount) AS total_sales,
  CASE
    WHEN SUM(amount) >= 5000 THEN 'Exceeds Target'
    WHEN SUM(amount) >= 3000 THEN 'Meets Target'
    WHEN SUM(amount) >= 1500 THEN 'Needs Improvement'
    ELSE 'Below Standard'
  END AS performance
FROM sales
GROUP BY rep_name;

-- Example 6: Date-based logic
SELECT
  order_id,
  order_date,
  CASE
    WHEN order_date >= CURRENT_DATE - INTERVAL '7 days' THEN 'This Week'
    WHEN order_date >= CURRENT_DATE - INTERVAL '30 days' THEN 'This Month'

```

```

        WHEN order_date >= CURRENT_DATE - INTERVAL '365 days' THEN 'This Year'
        ELSE 'Older'
    END AS recency
FROM orders
ORDER BY order_date DESC;

```

-- Example 7: Multiple conditions in one WHEN

```

SELECT
    product_name,
    category,
    price,
    CASE
        WHEN category = 'Electronics' AND price > 500 THEN 'Premium Tech'
        WHEN category = 'Electronics' AND price <= 500 THEN 'Budget Tech'
        WHEN category = 'Furniture' THEN 'Furniture'
        ELSE 'Other'
    END AS product_tier
FROM products;

```

CASE in SELECT

The most common use — compute a new derived column.

-- Example 8: CASE with NULL handling

```

SELECT
    customer_name,
    city,
    CASE
        WHEN city IS NULL THEN 'Location Unknown'
        WHEN country = 'USA' THEN city || ', USA'
        ELSE city || ', ' || country
    END AS display_location
FROM customers;

```

-- Example 9: Boolean flag using CASE

```

SELECT
    rep_name,
    amount,
    CASE WHEN amount > 1000 THEN 'High Value' ELSE 'Standard' END AS sale_tier,
    CASE WHEN region IS NULL THEN 1 ELSE 0 END AS is_unknown_region
FROM sales;

```

CASE in WHERE and HAVING

```

-- Example 10: Conditional filter using CASE in WHERE
-- Show orders: all if manager, only own if regular user
-- (simulated with a parameter)
SELECT order_id, customer_id, total_amount

```

```

FROM orders
WHERE
    CASE
        WHEN current_user = 'admin' THEN TRUE           -- admin sees all
        ELSE customer_id = 42                             -- others see only theirs
    END;

-- Example 11: CASE in HAVING
SELECT
    region,
    SUM(amount) AS revenue
FROM sales
GROUP BY region
HAVING
    CASE
        WHEN region = 'North' THEN SUM(amount) > 3000
        ELSE SUM(amount) > 1000
    END;

```

CASE in ORDER BY

CASE in ORDER BY enables custom sort sequences that don't follow alphabetical or numeric order.

```

-- Example 12: Custom priority sort
SELECT
    employee_name,
    CASE department_id
        WHEN 1 THEN 'Engineering'
        WHEN 2 THEN 'Marketing'
        WHEN 3 THEN 'Sales'
        WHEN 4 THEN 'HR'
        ELSE 'Unknown'
    END AS department_name,
    salary
FROM employees
ORDER BY
    CASE department_id
        WHEN 3 THEN 1   -- Sales first
        WHEN 1 THEN 2   -- Engineering second
        WHEN 2 THEN 3   -- Marketing third
        ELSE 4          -- rest last
    END,
    salary DESC;

-- Example 13: Sort NULL values to end
SELECT product_name, price
FROM products
ORDER BY
    CASE WHEN price IS NULL THEN 1 ELSE 0 END,
    price ASC;

```

CASE in GROUP BY

```
-- Example 14: Group by computed category
SELECT
  CASE
    WHEN salary < 60000 THEN 'Low'
    WHEN salary < 100000 THEN 'Mid'
    ELSE 'High'
  END AS salary_bracket,
  COUNT(*) AS employee_count,
  ROUND(AVG(salary), 0) AS avg_salary
FROM employees
GROUP BY
  CASE
    WHEN salary < 60000 THEN 'Low'
    WHEN salary < 100000 THEN 'Mid'
    ELSE 'High'
  END
ORDER BY avg_salary;
```

CASE in Aggregate Functions

Conditional aggregation — one of the most powerful CASE patterns.

```
-- Example 15: Count rows meeting different conditions
SELECT
  region,
  COUNT(*) AS total_sales,
  COUNT(CASE WHEN amount > 1500 THEN 1 END) AS high_value_sales,
  COUNT(CASE WHEN amount <= 1500 THEN 1 END) AS standard_sales
FROM sales
WHERE region IS NOT NULL
GROUP BY region;

-- Example 16: SUM with CASE — revenue by category bucket
SELECT
  rep_name,
  SUM(CASE WHEN amount > 1500 THEN amount ELSE 0 END) AS premium_revenue,
  SUM(CASE WHEN amount <= 1500 THEN amount ELSE 0 END) AS standard_revenue,
  SUM(amount) AS total_revenue
FROM sales
GROUP BY rep_name
ORDER BY total_revenue DESC;

-- Note: Equivalent FILTER syntax (preferred in PostgreSQL):
SELECT
  rep_name,
  SUM(amount) FILTER (WHERE amount > 1500) AS premium_revenue,
  SUM(amount) FILTER (WHERE amount <= 1500) AS standard_revenue,
```

```
SUM(amount) AS total_revenue
FROM sales
GROUP BY rep_name;
```

CASE for Pivot Tables

Transform rows to columns — a classic interview challenge.

ASCII Diagram — Pivot with CASE

Before (row-per-month):

rep	month	revenue
Alice	Jan	1500
Alice	Feb	800
Alice	Mar	1800
Bob	Jan	2200
...	...	...

After (pivot to columns):

rep	Jan	Feb	Mar
Alice	1500	800	1800
Bob	2200	1100	600
Carol	0	950	1400

-- Example 17: Pivot – monthly revenue per sales rep

```
SELECT
    rep_name,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 1 THEN amount ELSE 0 END) AS jan,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 2 THEN amount ELSE 0 END) AS feb,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 3 THEN amount ELSE 0 END) AS mar,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 4 THEN amount ELSE 0 END) AS apr,
    SUM(amount) AS total
FROM sales
GROUP BY rep_name
ORDER BY total DESC;
```

-- Example 18: Pivot – product sales by region

```
SELECT
    product,
    SUM(CASE WHEN region = 'North' THEN amount ELSE 0 END) AS north,
    SUM(CASE WHEN region = 'South' THEN amount ELSE 0 END) AS south,
    SUM(CASE WHEN region = 'East' THEN amount ELSE 0 END) AS east,
    SUM(CASE WHEN region = 'West' THEN amount ELSE 0 END) AS west,
    SUM(amount) AS total
FROM sales
WHERE region IS NOT NULL
GROUP BY product
ORDER BY total DESC;
```

Nested CASE

CASE expressions can be nested inside other CASE expressions.

```
-- Example 19: Two-level classification
SELECT
  product_name,
  category,
  price,
  CASE category
    WHEN 'Electronics' THEN
      CASE
        WHEN price > 500 THEN 'Premium Electronics'
        WHEN price > 100 THEN 'Mid-Range Electronics'
        ELSE 'Budget Electronics'
      END
    WHEN 'Furniture' THEN
      CASE
        WHEN price > 500 THEN 'Premium Furniture'
        ELSE 'Standard Furniture'
      END
    ELSE 'Other'
  END AS product_tier
FROM products;
```

ASCII Visual Diagrams

CASE Execution Flow

```
CASE
  WHEN condition1 → TRUE? —yes→ return result1 (stop evaluating)
                        condition1 → FALSE? —no→ continue
  WHEN condition2 → TRUE? —yes→ return result2 (stop evaluating)
                        condition2 → FALSE? —no→ continue
  WHEN condition3 → TRUE? —yes→ return result3 (stop evaluating)
                        condition3 → FALSE? —no→ continue
  ELSE —————→ return default (or NULL if no ELSE)
END
```

Conditional Aggregation Pattern

```
For each GROUP:
Row 1: amount=2000 → CASE matches → contributes to SUM
Row 2: amount=500 → CASE no match → contributes 0 or is excluded
Row 3: amount=1800 → CASE matches → contributes to SUM
Row 4: amount=300 → CASE no match → excluded
                                     ↓
GROUP SUM = 2000 + 1800 = 3800
```

COALESCE, NULLIF, GREATEST, LEAST

These are shorthand functions that could be written as CASE:

```

-- COALESCE: return first non-NULL value
COALESCE(a, b, c)
-- Equivalent:
CASE WHEN a IS NOT NULL THEN a WHEN b IS NOT NULL THEN b ELSE c END

-- Example 20: COALESCE for NULL replacement
SELECT
    customer_name,
    COALESCE(city, 'N/A') AS city,
    COALESCE(country, 'Unknown') AS country
FROM customers;

-- NULLIF: return NULL if both args are equal, else first arg
NULLIF(value, comparison_value)
-- Equivalent:
CASE WHEN value = comparison_value THEN NULL ELSE value END

-- Example 21: Avoid division by zero
SELECT
    product,
    SUM(amount) / NULLIF(SUM(units_sold), 0) AS revenue_per_unit
FROM sales
GROUP BY product;

-- GREATEST / LEAST: max/min across a list of values
-- Example 22: Ensure a minimum salary
SELECT
    employee_name,
    GREATEST(salary, 50000) AS adjusted_salary
FROM employees;

SELECT
    product_name,
    LEAST(price, 999.99) AS capped_price
FROM products;

```

Common Mistakes

Mistake 1: No ELSE clause — returns NULL unexpectedly

```

-- WRONG: if salary >= 200000, returns NULL (no ELSE)
SELECT employee_name,
    CASE WHEN salary < 50000 THEN 'Low' WHEN salary < 100000 THEN 'Mid' END
FROM employees;
-- Employees with salary >= 100000 get NULL silently

-- CORRECT:
CASE WHEN salary < 50000 THEN 'Low' WHEN salary < 100000 THEN 'Mid' ELSE 'High' END

```

Mistake 2: Wrong order of WHEN conditions

```
-- WRONG: ranges overlap – first match wins, so second WHEN never fires
CASE
  WHEN salary < 100000 THEN 'Mid'
  WHEN salary < 50000  THEN 'Low'   -- never reached for salary < 50000
END

-- CORRECT: narrowest range first
CASE
  WHEN salary < 50000  THEN 'Low'
  WHEN salary < 100000 THEN 'Mid'
  ELSE 'High'
END
```

Mistake 3: Mixing CASE types

```
-- WRONG: mixing incompatible types
CASE WHEN price > 100 THEN 'Expensive' ELSE price END -- text vs numeric

-- CORRECT: cast to a common type
CASE WHEN price > 100 THEN 'Expensive' ELSE price::TEXT END
```

Mistake 4: Using CASE in GROUP BY but not SELECT (or vice versa)

```
-- WRONG: GROUP BY uses different CASE expression than SELECT
SELECT CASE WHEN salary < 50000 THEN 'Low' ELSE 'High' END, COUNT(*)
FROM employees
GROUP BY CASE WHEN salary < 60000 THEN 'Low' ELSE 'High' END; -- different
threshold!
```

Best Practices

1. Always include an ELSE clause to handle unmatched conditions explicitly.
 2. Order WHEN clauses from most specific/narrow to most general.
 3. Use FILTER syntax instead of CASE inside aggregates when on PostgreSQL.
 4. Avoid deeply nested CASE — extract logic to a function or CTE.
 5. Use COALESCE instead of CASE for simple NULL-handling.
 6. Use NULLIF instead of CASE for division-by-zero protection.
 7. Keep CASE expressions in GROUP BY and SELECT identical (copy-paste or use ordinal).
-

Performance Considerations

```
-- 1. CASE itself is very lightweight (no I/O, computed in memory)
-- 2. CASE cannot use indexes on the CASE expression itself
--    but the WHEN conditions can use indexed columns in WHERE
```

```

-- 3. Conditional aggregation with CASE vs subquery:
-- CASE approach (single pass):
SELECT
    region,
    SUM(CASE WHEN amount > 1000 THEN amount ELSE 0 END) AS high_rev,
    SUM(amount) AS total_rev
FROM sales GROUP BY region;

-- Subquery approach (multiple passes – slower):
SELECT
    r.region,
    (SELECT SUM(amount) FROM sales WHERE region = r.region AND amount > 1000) AS
high_rev,
    (SELECT SUM(amount) FROM sales WHERE region = r.region) AS total_rev
FROM (SELECT DISTINCT region FROM sales WHERE region IS NOT NULL) r;
-- Use CASE – always single table scan!

-- 4. Generated columns for frequently computed CASE expressions
ALTER TABLE employees
    ADD COLUMN salary_band TEXT
    GENERATED ALWAYS AS (
        CASE
            WHEN salary < 50000 THEN 'Junior'
            WHEN salary < 100000 THEN 'Senior'
            ELSE 'Executive'
        END
    ) STORED;

```

Interview Questions & Answers

Q1: What is the difference between a simple CASE and a searched CASE?

A: A simple CASE compares a single expression against a series of values using equality (like a switch statement). A searched CASE evaluates a series of independent boolean conditions. Searched CASE is more flexible — it supports range checks, NULL tests, and complex predicates.

Q2: What does CASE return if no WHEN matches and there is no ELSE?

A: NULL. This is a common source of bugs. Always include an explicit ELSE clause.

Q3: How do you pivot rows to columns in SQL?

A: Use conditional aggregation with CASE inside aggregate functions: `SUM(CASE WHEN month = 'Jan' THEN amount ELSE 0 END) AS jan`. One CASE expression per output column, filtered by the pivot dimension.

Q4: What is the difference between COALESCE and CASE for NULL handling?

A: COALESCE(a, b, c) returns the first non-NULL value and is shorthand for a CASE. COALESCE is cleaner for simple NULL replacement. CASE is needed for complex conditional NULL handling.

Q5: Can CASE be used in ORDER BY?

A: Yes. CASE in ORDER BY enables custom sort sequences — for example, sorting by business priority rather than alphabetically.

Q6: How does NULLIF help prevent division by zero?

A: `NULLIF(denominator, 0)` returns NULL if the denominator is 0, and the actual value otherwise. Division by NULL returns NULL (not an error), which can then be handled by COALESCE.

Q7: Is CASE evaluated lazily (short-circuit)?

A: CASE stops evaluating at the first matching WHEN. However, PostgreSQL's planner does not guarantee strict left-to-right short-circuit evaluation of WHEN conditions in all cases — functions in WHEN conditions may be called out of order due to inlining. For safety-critical conditions (like CASE WHEN $x > 0$ THEN $1/x$), use explicit type handling or function barriers.

Q8: How do you use CASE to create a conditional GROUP BY?

A: Place the CASE expression in the GROUP BY clause to group by a computed category rather than raw column values. Ensure the exact same CASE expression appears in both SELECT and GROUP BY.

Hands-on Exercises

Exercise 1: Employee Salary Tier

Label each employee with a salary tier: Under \$50K = 'Entry', \$50K-\$80K = 'Mid', \$80K-\$120K = 'Senior', over \$120K = 'Executive'.

```
-- Solution:
SELECT
    employee_name,
    salary,
    CASE
        WHEN salary < 50000 THEN 'Entry'
        WHEN salary < 80000 THEN 'Mid'
        WHEN salary < 120000 THEN 'Senior'
        ELSE 'Executive'
    END AS salary_tier
FROM employees
ORDER BY salary DESC;
```

Exercise 2: Monthly Revenue Pivot

Create a pivot showing each sales rep's revenue for each month (Jan, Feb, Mar, Apr) in a single row.

```
-- Solution:
SELECT
    rep_name,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 1 THEN amount END) AS jan,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 2 THEN amount END) AS feb,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 3 THEN amount END) AS mar,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 4 THEN amount END) AS apr,
    SUM(amount) AS total
```

```
FROM sales
GROUP BY rep_name
ORDER BY total DESC NULLS LAST;
```

Exercise 3: Custom Sort Order

List products in this order: Electronics first, Furniture second, everything else last. Within each category, sort by price descending.

```
-- Solution:
SELECT product_name, category, price
FROM products
ORDER BY
    CASE category
        WHEN 'Electronics' THEN 1
        WHEN 'Furniture'   THEN 2
        ELSE 3
    END,
price DESC;
```

Exercise 4: Discount Calculator

Write a query that applies a discount based on order amount: orders >= \$300 get 10% off, orders >= \$150 get 5% off, all others no discount. Show the original amount, discount rate, and final price.

```
-- Solution:
SELECT
    order_id,
    total_amount,
    CASE
        WHEN total_amount >= 300 THEN '10%'
        WHEN total_amount >= 150 THEN '5%'
        ELSE 'None'
    END AS discount_rate,
    ROUND(
        total_amount * (1 - CASE
            WHEN total_amount >= 300 THEN 0.10
            WHEN total_amount >= 150 THEN 0.05
            ELSE 0
        END), 2
    ) AS final_price
FROM orders
ORDER BY total_amount DESC;
```

Advanced Notes

CASE in Window Functions

```
-- Use CASE as the window function input
SELECT
    rep_name,
    sale_date,
    amount,
    SUM(CASE WHEN amount > 1000 THEN amount ELSE 0 END)
      OVER (PARTITION BY rep_name ORDER BY sale_date) AS cumulative_premium
FROM sales;
```

CASE with ARRAY and JSON

```
-- CASE with JSON field extraction
SELECT
    order_id,
    CASE
        WHEN metadata->>'priority' = 'high' THEN 1
        WHEN metadata->>'priority' = 'medium' THEN 2
        ELSE 3
    END AS priority_rank
FROM orders
WHERE metadata IS NOT NULL;
```

Cross-references

- [02_aggregations.md](#) — FILTER clause as a cleaner alternative to CASE in aggregates
- [07_grouping_sets_rollup_cube.md](#) — multi-level aggregation without manual CASE pivots
- [01_window_functions_intro.md](#) (04_Advanced_SQL) — CASE inside window function expressions
- [06_database_design/](#) — Generated columns for stored CASE computations